More specifically, we compute the **indirect effect** (IE) of a specific mediating component by quantifying its contribution in skewing $\mathbb{P}$ towards the correct result. Consider a generic activation variable $z \in \{m_1^{(1)}, \ldots, m_t^{(L)}, a_1^{(1)}, \ldots, a_t^{(L)}\}$. We denote the model's output probability following an intervention on z as $\mathbb{P}_z^*$. Then, we compute the IE as:

$$\mathrm{IE}(z) = \frac{1}{2}\left[\frac{\mathbb{P}_z^*(r) - \mathbb{P}(r)}{\mathbb{P}(r)} + \frac{\mathbb{P}(r') - \mathbb{P}_z^*(r')}{\mathbb{P}_z^*(r')}\right] \quad (2)$$

where the two terms in the sum represent the relative change in the probability assigned by the model to r and r', caused by the intervention performed. The larger the measured IE, the larger the contribution of component z in shifting probability mass from the clean-run result r' to result r corresponding to the alternative input p_1.[3]

We additionally measure the mediation effect of each component with respect to the operation f_O. We achieve this by fixing the operands and changing the operator across the two input prompts. More formally, in step 1, we sample a list of operands N and two operators f_O and f'_O. Then, we generate two prompts $p_1 = p(N, f_O)$ and $p_2 = p(N, f'_O)$ (e.g., "*What is the sum of 11 and 7?*" and "*What is the product of 11 and 7?*"). Finally, we carry out the procedure in steps 2–4.

3.3 Experimental Setup

We present the results of our analyses in the main paper for GPT-J (Wang and Komatsuzaki, 2021), a 6B-parameter pre-trained LM (Gao et al., 2020). Additionally, we validate our findings on Pythia 2.8B (Biderman et al., 2023), LLaMA 7B (Touvron et al., 2023), and Goat, a version of LLaMA fine-tuned on arithmetic tasks (Liu and Low, 2023). We report the detailed results for these models in Appendix J.

In our experiments, we focus on two- and three-operand arithmetic problems. Similar to previous work (Razeghi et al., 2022; Karpas et al., 2022), for single-operator two-operand queries, we use a set of six diverse templates representing a question involving each of the four arithmetic operators. For the three-operand queries, we use one template for each of the 29 possible two-operator combinations. Details about the templates are reported in Appendix A. In the bi-variate case, for each of the four operators $f_O \in \{+, -, \times, \div\}$ and for each of the templates, we generate 50 pairs of prompts by sampling two pairs of operands $(n_1, n_2) \in \mathcal{S}^2$ and $(n'_1, n'_2) \in \mathcal{S}^2$, where $\mathcal{S} \subset V \cap \mathbb{N}$. For the operand-related experiment, we sample (n_1, n_2) and a second operation f'_O. In both cases, we ensure that the result r falls within $\mathcal{S}$.[4] In the three-operand case, we generate 15 pairs of prompts for each of the 29 templates, following the same procedure. In order to ensure that the model achieves a meaningful task performance, we use a two-shot prompt in which we include two exemplars of question-answer for the same operation that is being queried. We report the accuracy results in Appendix B.

4 Causal Effects on Arithmetic Queries

Our analyses address the following question:

Q1 What are the components of the model that mediate predictions involving arithmetic computations?

We address this question by first studying the flow of information throughout the model by measuring the effect of each component (MLP and attention block) at each point of the input sequence for two-operand queries (§4.1). Then, we distinguish between model components that carry information about the result and about the operands of the arithmetic computations (§4.2 and §4.3). Finally, we consider queries involving three operands (§4.4) and present a measure to quantify the changes in information flow (§4.5).

4.1 Tracing the Information Flow

We measure the indirect effect of each MLP and attention block at different positions along the input sequence. The output of these modules can be seen as new information being incorporated into the residual stream. This new information can be produced at any point of the sequence and then conveyed to the end of the sequence for the prediction of the next token. By studying the IE at different locations within the model, we can identify the modules that generate new information relevant to the model's prediction. The results are reported in Figures 3a and 3b for MLP and attention, respectively.

[3] As an alternative metric to quantify the IE, we experiment using the difference in log probabilities (Appendix I). The results obtained with the two metrics show consistency and lead to the same conclusions.

[4] Unless otherwise specified, we use $\mathcal{S} = \{1, 2, \ldots, 300\}$, as larger integers get split into multiple tokens by the tokenizer.

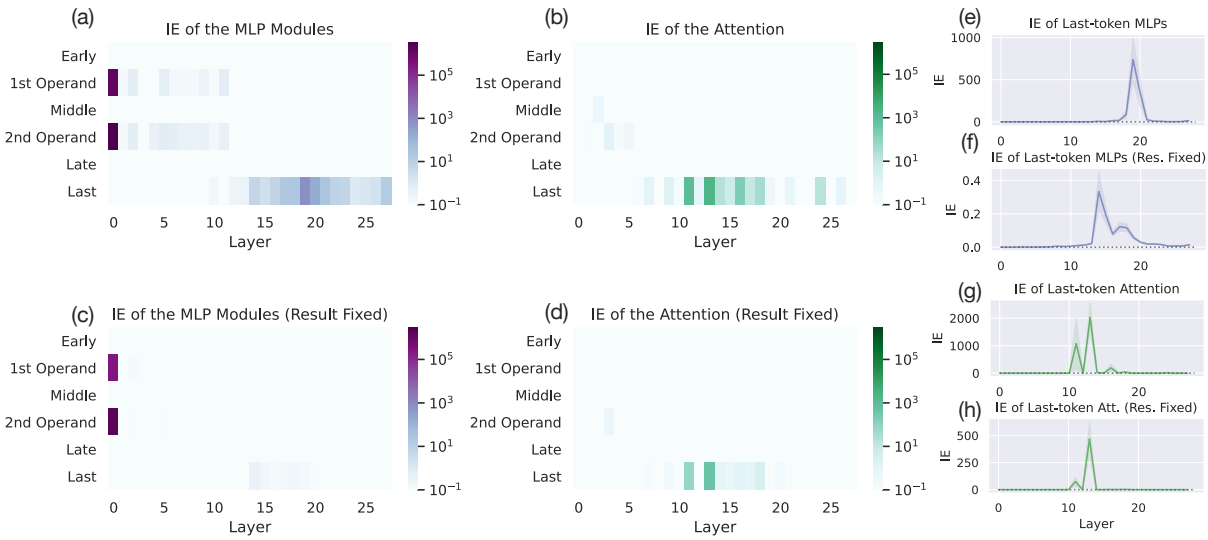

Figure 3: Indirect effect (IE) measured within GPT-J. Figures (a) and (b) illustrate the flow of information related to both the operands and the result of the queries, while the effect displayed in Figures (c) and (d) is related to the operands only (the result is kept unchanged). Figures (e–h) show a re-scaled visualization of the effects at the last token for each of the four heatmaps (a–d). The difference in the effect registered for the MLPs at layers 15–25 between figures (a) and (c) illustrates the role of these components in producing result-related information.

Our analysis reveals four primary activation sites: the MLP module situated at the first layer corresponding to the tokens of the two operands; the intermediate attention blocks at the last token of the sequence; and the MLP modules in the middle-to-late layers, also located at the last token of the sequence. It is expected to observe a high effect for the first MLPs associated with the tokens that vary (i.e., the operands), as such modules are likely to affect the representation of the tokens, which is subsequently used for the next token prediction. On the other hand, of particular interest is the high effect detected at the attention modules in layers 11–18 and in the MLPs around layer 20.

As for the flow of information tied to the operator, the activations display a parallel pattern: high effect is registered at early MLPs associated with the operator tokens and at the same last-token MLP and attention locations. We report the visualization of the operator-related results in Appendix C.

A possible explanation of the model's behavior on this task is that the attention mechanism facilitates the propagation of operand- and operator-related information from the first layers early in the sequence to the last token. Here, this information is processed by the MLP modules, which incorporate the information about the result of the computation in the residual stream. This hypothesis aligns with the existing theory that attributes the responsibility of moving and extracting information within Transformer-based models to the attention mechanism (Elhage et al., 2021; Geva et al., 2023), while the feed-forward layers are associated with performing computations, retrieving facts and information (Geva et al., 2022; Din et al., 2023; Meng et al., 2022). We test the validity of this hypothesis in the following section.

4.2 Operand- and Result-related Effects

Our objective is to verify whether the contribution to the model's prediction of each component measured in Figures 3a and 3b is due to (*1*) the component representing information related to the operands, or (*2*) the component encoding information about the result of the computation. To this end, we formulate a variant of our previous experimental procedure. In particular, we condition the sampling of the second pair of operands (n_1', n_2') on the constraint $r = r'$. That is, we generate the two input questions p_1 and p_2, such that their result is the same (e.g., "*What is the sum of 25 and 7?*" and "*What is the sum of 14 and 18?*"). In case number (*1*), we would expect a component to have high IE both in the result-varying setting and when $r = r'$, as the operands are modified in both scenarios. In case (*2*), we expect a subset of the model to have a large effect when the operands are sampled without constraints but a low effect for the fixed-result setting.

We report the results in Figure 3c and 3d. By

comparing Figures 3a and 3c, two notable observations emerge. First, the high effect in the early layers corresponding to the operand tokens is observed in both the result-preserving and the result-varying scenarios. Second, the last-token mid-late MLPs that lead to a high effect on the model's prediction following a result change, dramatically decrease their effect on the model's output in the result-preserving setting, as described in scenario (*2*). These observations point to the conclusion that the MLP blocks around layer 20 incorporate result-relevant information. As for the contribution of the attention mechanism (Figures 3b and 3d), we do not observe a substantial difference in the layers with the highest IE between the two settings, which aligns this scenario to the description of case (*1*). These results are consistent with our hypothesis that operand-related information is transferred by the attention mechanism to the end of the sequence and then processed by the MLPs to obtain the result of the computation.

4.3 Zooming in on the Last Token

In Figures 3e–3h, we show a re-scaled version of the IE measurements for the layers at the end of the input sequence. While the large difference in magnitude was already evident in the previously considered visualizations, in Figures 3e and 3f we notice that the MLPs with the highest effect in the two settings differ: the main contribution to the model's output when the results are not fixed is given by layers 19 and 20, while in the result-preserving setting the effect is largest at layers 14-18. For the attention (Figures 3g and 3h), we do not observe a significant change in the shape of the curve describing the IE across different layers, with layer 13 producing the largest contribution. We interpret this as additional evidence indicating that the last-token MLPs at layers 19-20 encode information about r, while the attention modules carry information related to the operands.

4.4 Three-operand Queries & Fine-tuning

We extend our analyses by including three-operand arithmetic queries such as "*What is the difference between n_1 and the ratio between n_2 and n_3?*". Answering correctly this type of questions represents a challenging task for pre-trained language models, and we observe poor accuracy (below 10%) with GPT-J. Thus, we opt for fine-tuning the model on a small set of three-operand queries. The model that we consider for this analysis is Pythia 2.8B, as its smaller size allows for less computationally demanding training than the 6B-parameter GPT-J. After fine-tuning, the model attains an accuracy of ∼40%. We provide the details about the training procedure in Appendix F.

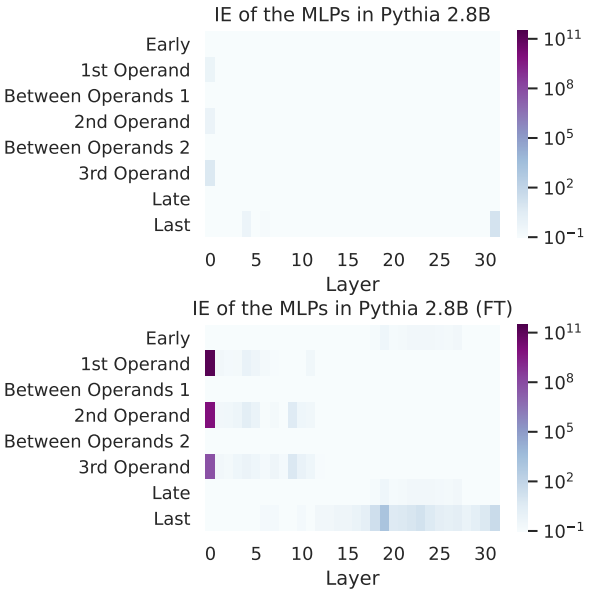

Figure 4: Indirect effect (IE) on three-operand queries for different MLP modules in Pythia 2.8B before and after fine-tuning. The effect produced by the last-token mid-late MLP activation site emerges with fine-tuning. Results for the attention are reported in Appendix J.

We carry out the experimental procedure as in Section 4.1. In particular, we compare the information flow in the MLPs of the model before and after fine-tuning (Figure 4). In the non-fine-tuned version of the model, the only relevant activation site, besides the early layers at the operand tokens, is the very last layer at the last token. In the fine-tuned model, on the other hand, we notice the emergence of the mid-late MLP activation site that was previously observed in the two-operand setting.

4.5 Quantifying the Change of the Information Flow

Denote the set of MLPs in the model by $\mathcal{M}$. We define the relative importance (RI) of a specific subset $\mathcal{M}^* \subseteq \mathcal{M}$ of MLP modules as

$$\mathrm{RI}(\mathcal{M}^*) = \frac{\sum_{m \in \mathcal{M}^*} \log(\mathrm{IE}(m) + 1)}{\sum_{m \in \mathcal{M}} \log(\mathrm{IE}(m) + 1)}. \quad (3)$$

In order to quantitatively show the difference in the activation sites observed in Figure 3, we compute the RI measure for the set

$$\mathcal{M}_{-1}^{\text{late}} = \{m_{-1}^{(\lfloor L/2 \rfloor)}, m_{-1}^{(\lfloor L/2 \rfloor + 1)}, \ldots, m_{-1}^{(L)}\},$$